



Web test helpers

Overview

After the `"@odoo/hoot"` module, the second most-solicited module in test files should be `"@web/./tests/web_test_helpers"` .

This module contains all the helpers that combine the low-level helpers provided by Hoot, with all the most common features that are used in tests in Odoo.

These helpers are many, and this section of the documentation will only highlight the most common ones, and the way they interact with one another.

For a full list of available helpers, you may refer to the [web_test_helpers file](#)

Mock environment

The `makeMockEnv` helper is the lowest helper that can spawn an `env` .

It will take care of

- creating the `env` object itself, pre-configured with all the required properties for the proper functioning of web components, such as `getTemplate` or `translateFn` ;
- spawning a `MockServer` (if one did not exist already for that test);
- starting all registered [services](#), and awaiting until they are all ready;
- initiating other features that are not tied to a service, such as the `web` router ;
- guaranteeing the teardown of all the features in its setup at the end of the test.

This method is great for testing low-level features, such as [services](#) that are not tied to a [Component](#)

```
// Can be further configured, but is already packed with all the necessary stuff  
const env = await makeMockEnv();  
  
expect(env.isSmall).toBe(false);
```

Note

Example

- [DateTime input tests](#)
- [Name service tests](#)

Mounting components

Instantiating and appending [components](#) to the DOM is meant to be easy, through the use of the `mountWithCleanup` helper. It will prepare an `env` internally (if one does not exist yet), which in turn also makes sure that a `MockServer` is running.

It takes a `Component` class as its first argument, and an *optional* parameters second argument, used to specify `props` or a custom `target` :

```
await mountWithCleanup(Checkbox, {
  props: {
    value: false
  },
});
```

This helper will return the active `Component` instance.

Important

It is generally *ill-advised* to retrieve the `Component` instance to directly interact with it or to perform assertions on its internal variables. The only “accepted” use cases are when the `Component` is displaying hard-to-retrieve information in the DOM, such as graphs in a [canvas](#). For most cases, it is highly preferred to query derived information in the DOM.

Example

- [Checkbox tests](#)
- [Popover tests](#)
- [DateTimePicker tests](#)

and the correct properties.

For that purpose, web test helpers export a `mountView` helper, taking a `parameters` object determining the `view` type, `resModel`, and other optional properties such as an XML `arch`:

```
// Resolves when the view is fully ready
await mountView({
  type: "list",
  resModel: "res.partner",
  arch: /* xml */ `
    <list>
      <field name="display_name" />
    </list>
  `,
});
```

Like the previous helpers on top of which `mountView` is built, it will ensure that both an `env` and a `MockServer` are running for the current test.

Note

Like [mountWithCleanup](#), it is *NOT* recommended to retrieve the returned [View](#) component instance. It can however be done, for cases like the [Graph view](#).

Example

- [Calendar view tests](#)
- [Graph view tests](#)
- [Kanban view tests](#)

Interacting with components

Hoot provides helpers to interact with the DOM (e.g. `click`, `press`, etc.). However, these helpers present 2 issues when interacting with more complex components:

1. helpers try to interact instantly, while sometimes the element has yet to be appended to the document (in an unknown amount of time);

```
// Edit record name
await click(".o_field_widget[name=name]");
await edit("Gaston Lagaffe");

// Potential error 1: button may not be in the DOM yet
await click(".btn:contains(Save)");

// Potential error 2: view is not yet updated
expect(".o_field_widget[name=name]").toHaveText("Gaston Lagaffe");
```

With these constraints in mind, web test helpers provide the `contains` helper:

```
// Combines 'click' + 'edit' + 'animationFrame' calls
await contains(".o_field_widget[name=name]").edit("Gaston Lagaffe");
// Waits for (at least) a full animation frame after the click
await contains(".btn:contains(Save)").click();
expect(".o_field_widget[name=name]").toHaveText("Gaston Lagaffe");
```

This approach, while seemingly drifting a bit further away from the concept of “unit testing”, is still a nice and convenient way to test more complex units such as [views](#), the [WebClient](#), or interactions between couples of [services](#) and components.

It should however not become the default for all interactions, as some of them still need to happen *precisely* within a given time frame, which is a concept completely ignored by `contains`.

Note

Most helpers in Hoot are available as methods of a `contains` instance, with (generally) the same shape and API.

